JAVA
TECH COMMUNITY

SERIES #1: CRACK MNC JAVA INTERVIEW

About Series #1: Crack MNC Java Interview

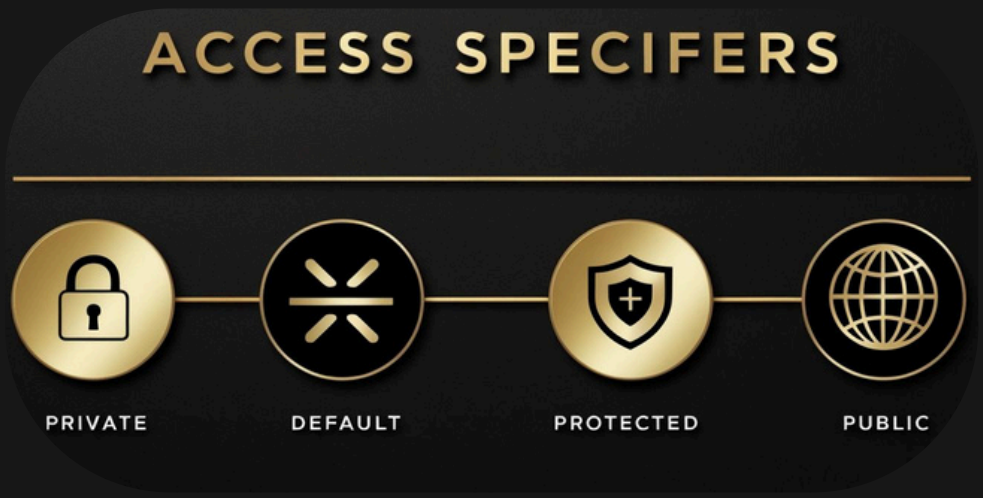
📘 **Who Is It For:**

- This series is designed for freshers, graduates, and professionals with 1-5 years of experience.
- It focuses on FAQ questions relevant to major MNC interviews.

🚀 **How to Use This Guide:**

- Use this to learn how real-time interviews ask questions 🎯.
- Focus on applying concepts and showcasing your project experience 💻🚀.

DAY #5: Java Access Modifiers/specifiers Interview Guide



What Are Access Modifiers in Java?

🔐 **Access Modifiers** in Java are keywords that define the **scope** and **visibility** of classes, methods, and variables.
The four types are:

- 🔒 **Private**
- 📦 **Default**
- 🛡️ **Protected**
- 🌍 **Public**

Modifier	👨‍🏫 Class	📦 Package	🧩 Subclass	🌐 Global
🔒 **Private**	✓ Yes	✗ No	✗ No	✗ No
📦 **Default**	✓ Yes	✓ Yes	✗ No	✗ No
🛡️ **Protected**	✓ Yes	✓ Yes	✓ Yes	✗ No
🌍 **Public**	✓ Yes	✓ Yes	✓ Yes	✓ Yes

DAY #5: Java Access Modifiers/specifiers Interview Guide

1. **Private (🔒)**: Accessible only within the same class; used for strict encapsulation.
2. **Default (📦)**: Accessible within the same package; no modifier keyword is needed.
3. **Protected (🛡️)**: Accessible within the same package and subclasses; supports inheritance.
4. **Public (🌍)**: Accessible from anywhere; provides global visibility.

Details with code example

🔒 Private

- **Scope:** Inside the same class only.
- **Use:** Encapsulation, data hiding.
- **Example:**

java Copy

```java
class Example {
    private int data = 42; // 🔒 Private
    private void display() {
        System.out.println("Data: " + data);
    }
}
```

📦 Default

- **Scope:** Within the same package.
- **Use:** Package-level access.
- **Example:**

java Copy

```java
class Example {
    int data = 42; // 📦 Default
    void display() {
        System.out.println("Data: " + data);
    }
}
```

Details with code example

Protected

- **Scope:** Accessible within the same package and subclasses.
- **Use:** Inheritance and package-level visibility.
- **Example:**

java Copy

```java
class Parent {
    protected int data = 42; // Protected
}
class Child extends Parent {
    void display() {
        System.out.println("Data: " + data);
    }
}
```

Public

- **Scope:** Accessible everywhere.
- **Use:** Global access for APIs or libraries.
- **Example:**

java Copy

```java
public class Example {
    public int data = 42; // Public
    public void display() {
        System.out.println("Data: " + data);
    }
}
```

case study-based code snippet questions focused on access modifiers

Can We Have a 🔒 Private Constructor in Java?

✔️ Yes!
A **private constructor** is used to restrict the instantiation of a class, typically in **singleton design patterns**.

💻 **Example:**

java Copy

```java
public class Singleton {
    private Singleton() { // 🔒 Private constructor
        // Constructor logic
    }

    private static Singleton instance = new Singleton(); // 🛠️ Singleton instance

    public static Singleton getInstance() { // 🌍 Public accessor
        return instance;
    }
}
```

Which Is the 📦 Default Access Modifier in Java?

- For top-level classes, only **public** 🌍 and **default** 📦 access modifiers are allowed.
- 🔒 **Private** and 🛡️ **Protected** cannot be used with top-level classes.

Can We Declare an Abstract Method as 🔒 Private?

❌ **No!**
Abstract methods cannot be private because they are meant to be overridden in subclasses, and private access 🔒 restricts visibility.

Why Are 🔐 Access Modifiers Used?

🔐 Access modifiers serve the following purposes:

1. 🛡️ **Encapsulation**: Hiding implementation details.
2. 🟥 **Security**: Restricting unauthorized access.
3. 🔄 **Reusability**: Controlling how code components are accessed.

Task for readers:

These are the most FAQ OOP basics! 🚀

- 💻 Explore More: Dive deeper into OOP FAQs, practice, and try small code snippets for each concept 💻. We'll share detailed code snippets on the community channel soon. For now, we want you to explore and learn! 🚀
- 🛠️ For Experienced Professionals: Reflect on how you've implemented these concepts in your current project.
- 🎓 For College Students & Graduates: Relate these concepts to your syllabus and try implementing small projects.

Pro Tip: Think Big, Start Small! 🌟 Let's grow together!

Note:

- This document is meant for brush-ups key concepts & Top FAQ.
- We always recommend working on projects and practicing hands-on coding to solidify your concepts. 💻✨